

Cómo nombrar y llamar procedimientos dentro de un paquete en Oracle

Buena pregunta, y es algo fundamental para trabajar en entornos empresariales. Vamos por partes.

Concepto básico: el paquete y sus procedimientos

Un **paquete** es como una **carpeta** donde guardas procedimientos y funciones relacionados. Tiene dos partes:

1. **Especificación (spec) o cabecera:** Es el "índice" del paquete. Aquí declaras qué procedimientos y funciones contiene.
2. **Cuerpo (body):** Es la implementación real de cada procedimiento y función.

Regla de oro: El nombre del procedimiento **NO repite el nombre del paquete**. Se llama como la acción que hace.

Convenciones de nombres (las que usan las empresas)

Elemento	Prefijo/Convención	Ejemplo
Esquema	Nombre de la empresa o usuario	MI_EMPRESA
Paquete	PKG_ + área funcional	PKG_PROCESOS_BLOQUEOS
Procedimiento	PR_ + acción	PR_BLOQUEAR_PUNTO_VENTA
Función	FN_ + qué devuelve	FN_CALCULAR_MORA
Parámetro de entrada	p_ + nombre	p_id_punto_venta
Parámetro de salida	p_ + nombre + _out	p_id_arreglo_out
Variable local	v_ + nombre	v_total_mora
Constante	c_ + nombre	c_ESTADO_ACTIVO



Ejemplo completo: paquete con procedimientos

1. Especificación del paquete (la cabecera)

sql

```
CREATE OR REPLACE PACKAGE MI_EMPRESA.PKG_PROCESOS_BLOQUEOS AS
```

```
    -- Procedimiento SIN parámetros
```

```
    PROCEDURE PR_BLOQUEAR_TODOS;
```

```
    -- Procedimiento CON parámetros
```

```
    PROCEDURE PR_BLOQUEAR_PUNTO_VENTA (  
        p_id_punto_venta IN NUMBER,  
        p_usuario_app    IN VARCHAR2,  
        p_motivo          IN VARCHAR2 DEFAULT 'Bloqueo manual'  
    );
```

```
    -- Procedimiento CON parámetros de entrada y salida
```

```
    PROCEDURE PR_CREAR_ARREGLO_PAGO (  
        p_id_punto_venta IN NUMBER,  
        p_id_cliente     IN NUMBER,  
        p_monto_acordado IN NUMBER,  
        p_num_cuotas     IN NUMBER,  
        p_usuario_app    IN VARCHAR2,  
        p_id_arreglo_out OUT NUMBER,  
        p_mensaje_out    OUT VARCHAR2  
    );
```

```
    -- Función (devuelve un valor)
```

```
    FUNCTION FN_CALCULAR_DIAS_MORA (  
        p_id_punto_venta IN NUMBER  
    ) RETURN NUMBER;
```

```
END PKG_PROCESOS_BLOQUEOS;
```

```
/
```

2. Cuerpo del paquete (la implementación)

sql

```
CREATE OR REPLACE PACKAGE BODY MI_EMPRESA.PKG_PROCESOS_BLOQUEOS AS

-- =====
-- Procedimiento SIN parámetros
-- =====

PROCEDURE PR_BLOQUEAR_TODOS IS
    v_contador NUMBER := 0;
BEGIN
    FOR r IN (
        SELECT pv.id_punto_venta
        FROM punto_venta pv
        JOIN cliente c ON c.id_punto_venta = pv.id_punto_venta
        JOIN servicio s ON s.id_cliente = c.id_cliente
        JOIN factura f ON f.id_servicio = s.id_servicio
        WHERE f.estado IN ('VENCIDA','EN_MORA')
        AND f.saldo_pendiente > 0
        AND pv.estado = 'ACTIVO'
        GROUP BY pv.id_punto_venta
    ) LOOP
        UPDATE punto_venta
        SET estado = 'INACTIVO'
        WHERE id_punto_venta = r.id_punto_venta;

        v_contador := v_contador + 1;
    END LOOP;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('PV bloqueados: ' || v_contador);
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END PR_BLOQUEAR_TODOS;

-- =====
-- Procedimiento CON parámetros
-- =====
```

```

PROCEDURE PR_BLOQUEAR_PUNTO_VENTA (
    p_id_punto_venta IN NUMBER,
    p_usuario_app    IN VARCHAR2,
    p_motivo          IN VARCHAR2 DEFAULT 'Bloqueo manual'
) IS
    v_estado VARCHAR2(10);
BEGIN
    SELECT estado INTO v_estado
    FROM punto_venta
    WHERE id_punto_venta = p_id_punto_venta
    FOR UPDATE;

    IF v_estado = 'INACTIVO' THEN
        RAISE_APPLICATION_ERROR(-20010, 'El PV ya está bloqueado.');
```

END IF;

```

    UPDATE punto_venta
        SET estado = 'INACTIVO'
    WHERE id_punto_venta = p_id_punto_venta;

    INSERT INTO historial_bloqueo_pv (
        id_historial, id_punto_venta, estado_anterior, estado_nuevo,
        motivo, usuario_app, origen
    ) VALUES (
        seq_historial_bloqueo.NEXTVAL, p_id_punto_venta,
        'ACTIVO', 'INACTIVO', p_motivo, p_usuario_app, 'MANUAL'
    );

    COMMIT;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20011, 'PV no encontrado.');
```

WHEN OTHERS THEN

```

        ROLLBACK;
        RAISE;
END PR_BLOQUEAR_PUNTO_VENTA;
```

-- =====

-- Procedimiento CON parámetros IN y OUT

-- =====

```

PROCEDURE PR_CREAR_ARREGLO_PAGO (
```

```

p_id_punto_venta    IN  NUMBER,
p_id_cliente        IN  NUMBER,
p_monto_acordado    IN  NUMBER,
p_num_cuotas        IN  NUMBER,
p_usuario_app       IN  VARCHAR2,
p_id_arreglo_out    OUT NUMBER,
p_mensaje_out       OUT VARCHAR2
) IS
BEGIN
    IF p_monto_acordado IS NULL OR p_monto_acordado <= 0 THEN
        p_mensaje_out := 'Monto inválido';
        RETURN;
    END IF;

    p_id_arreglo_out := seq_arreglo_pago.NEXTVAL;

    INSERT INTO arreglo_pago (
        id_arreglo, id_cliente, fecha_acuerdo,
        monto_acordado, num_cuotas, estado
    ) VALUES (
        p_id_arreglo_out, p_id_cliente, SYSDATE,
        p_monto_acordado, p_num_cuotas, 'ACTIVO'
    );

    UPDATE punto_venta
        SET estado = 'ACTIVO'
    WHERE id_punto_venta = p_id_punto_venta
        AND estado = 'INACTIVO';

    INSERT INTO historial_bloqueo_pv (
        id_historial, id_punto_venta, estado_anterior, estado_nuevo,
        motivo, usuario_app, origen
    ) VALUES (
        seq_historial_bloqueo.NEXTVAL, p_id_punto_venta,
        'INACTIVO', 'ACTIVO',
        'Desbloqueo por arreglo ID: ' || p_id_arreglo_out,
        p_usuario_app, 'MANUAL'
    );

    COMMIT;

    p_mensaje_out := 'Arreglo creado exitosamente';

```

```

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        p_mensaje_out := 'Error: ' || SQLERRM;
        RAISE;
END PR_CREAR_ARREGLO_PAGO;

-- =====
-- Función que devuelve un valor
-- =====

FUNCTION FN_CALCULAR_DIAS_MORA (
    p_id_punto_venta IN NUMBER
) RETURN NUMBER IS
    v_dias NUMBER;
BEGIN
    SELECT NVL(MAX(TRUNC(SYSDATE - f.fecha_vencimiento)), 0)
        INTO v_dias
    FROM cliente c
    JOIN servicio s ON s.id_cliente = c.id_cliente
    JOIN factura f ON f.id_servicio = s.id_servicio
    WHERE c.id_punto_venta = p_id_punto_venta
        AND f.estado IN ('VENCIDA', 'EN_MORA')
        AND f.saldo_pendiente > 0;

    RETURN v_dias;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0;
END FN_CALCULAR_DIAS_MORA;

END PKG_PROCESOS_BLOQUEOS;
/

```

Cómo llamar los procedimientos

La sintaxis general es:

text

```
ESQUEMA.NOMBRE_PAQUETE.NOMBRE_PROCEDIMIENTO(parámetros);
```

A Desde SQL Developer / PL/SQL Developer (bloque anónimo)

1. Llamar un procedimiento SIN parámetros

```
sql  
  
BEGIN  
    MI_EMPRESA.PKG_PROCESOS_BLOQUEOS.PR_BLOQUEAR_TODOS;  
END;  
/
```

2. Llamar un procedimiento CON parámetros (forma posicional)

El orden de los parámetros debe coincidir con la declaración:

```
sql  
  
BEGIN  
    MI_EMPRESA.PKG_PROCESOS_BLOQUEOS.PR_BLOQUEAR_PUNTO_VENTA(  
        10000,  
        'USUARIO_JUAN',  
        'Cliente moroso'  
    );  
END;  
/
```

3. Llamar un procedimiento CON parámetros (forma nombrada, recomendada)

Es más clara y no importa el orden:

```
sql  
  
BEGIN  
    MI_EMPRESA.PKG_PROCESOS_BLOQUEOS.PR_BLOQUEAR_PUNTO_VENTA(  
        p_id_punto_venta => 10000,  
        p_usuario_app    => 'USUARIO_JUAN',  
        p_motivo          => 'Cliente moroso'  
    );  
END;
```

/

4. Llamar un procedimiento con parámetros IN y OUT

sql

DECLARE

```
v_id_arreglo NUMBER;  
v_mensaje     VARCHAR2(200);
```

BEGIN

```
MI_EMPRESA.PKG_PROCESOS_BLOQUEOS.PR_CREAR_ARREGLO_PAGO(  
    p_id_punto_venta => 10000,  
    p_id_cliente     => 1,  
    p_monto_acordado => 300000,  
    p_num_cuotas     => 6,  
    p_usuario_app    => 'USUARIO_JUAN',  
    p_id_arreglo_out => v_id_arreglo,  
    p_mensaje_out    => v_mensaje  
);
```

```
DBMS_OUTPUT.PUT_LINE('ID Arreglo: ' || v_id_arreglo);
```

```
DBMS_OUTPUT.PUT_LINE('Mensaje: ' || v_mensaje);
```

END;

/

5. Llamar una función (¡se puede llamar dentro de un SELECT!)

sql

-- En un bloque PL/SQL

DECLARE

```
v_dias NUMBER;
```

BEGIN

```
v_dias := MI_EMPRESA.PKG_PROCESOS_BLOQUEOS.FN_CALCULAR_DIAS_MORA(10000);
```

```
DBMS_OUTPUT.PUT_LINE('Días de mora: ' || v_dias);
```

END;

/

-- O directamente en un SELECT

```
SELECT MI_EMPRESA.PKG_PROCESOS_BLOQUEOS.FN_CALCULAR_DIAS_MORA(10000) AS dias  
FROM dual;
```

B Desde Oracle Forms (botones del formulario)

1. Llamar un procedimiento SIN parámetros

```
sql

-- Trigger WHEN-BUTTON-PRESSED del botón "Bloquear Todos"
BEGIN
    MI_EMPRESA.PKG_PROCESOS_BLOQUEOS.PR_BLOQUEAR_TODOS;
    MESSAGE('Todos los PV han sido bloqueados.');
```

END;

2. Llamar un procedimiento CON parámetros

```
sql

-- Trigger WHEN-BUTTON-PRESSED del botón "Bloquear"
DECLARE
    v_id_pv NUMBER := :BLK_PV.ID_PUNTO_VENTA;
BEGIN
    IF v_id_pv IS NULL THEN
        MESSAGE('Debe seleccionar un punto de venta.');
```

RAISE FORM_TRIGGER_FAILURE;

END IF;

MI_EMPRESA.PKG_PROCESOS_BLOQUEOS.PR_BLOQUEAR_PUNTO_VENTA(

 p_id_punto_venta => v_id_pv,

 p_usuario_app => :GLOBAL.usuario_app,

 p_motivo => :BLK_ACCION.IT_OBSERVACIONES

);

MESSAGE('Punto de venta bloqueado.');

:BLK_PV.ESTADO := 'INACTIVO';

SYNCHRONIZE;

EXCEPTION

 WHEN OTHERS THEN

 MESSAGE('Error: ' || SQLERRM);

 RAISE FORM_TRIGGER_FAILURE;

END;

3. Llamar un procedimiento con parámetros OUT

```
sql

-- Trigger WHEN-BUTTON-PRESSED del botón "Arreglo de Pago"
DECLARE
    v_id_arreglo NUMBER;
    v_mensaje     VARCHAR2(200);
BEGIN
    MI_EMPRESA.PKG_PROCESOS_BLOQUEOS.PR_CREAR_ARREGLO_PAGO(
        p_id_punto_venta => :BLK_PV.ID_PUNTO_VENTA,
        p_id_cliente     => :BLK_PV.IT_ID_CLIENTE_MORA,
        p_monto_acordado => :BLK_ARREGLO.IT_MONTO,
        p_num_cuotas     => :BLK_ARREGLO.IT_CUOTAS,
        p_usuario_app    => :GLOBAL.usuario_app,
        p_id_arreglo_out => v_id_arreglo,
        p_mensaje_out    => v_mensaje
    );

    MESSAGE(v_mensaje || ' ID: ' || v_id_arreglo);
    HIDE_WINDOW('WIN_ARREGLO');
    GO_BLOCK('BLK_PV');
    EXECUTE_QUERY;
EXCEPTION
    WHEN OTHERS THEN
        MESSAGE('Error: ' || SQLERRM);
        RAISE FORM_TRIGGER_FAILURE;
END;
```

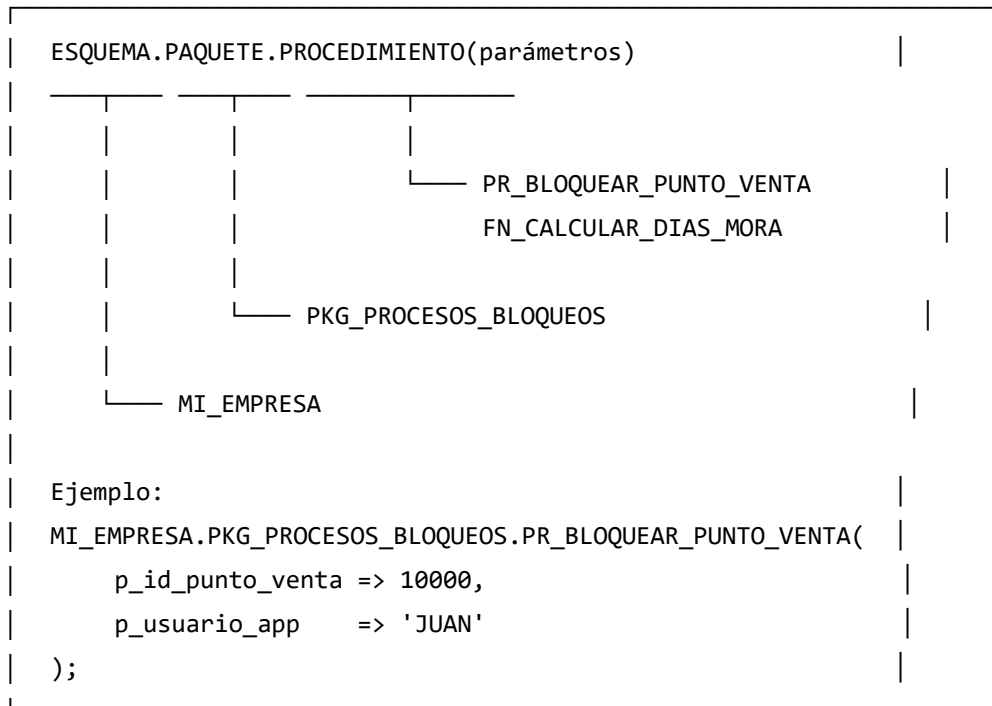
4. Llamar una función desde Forms

```
sql

DECLARE
    v_dias NUMBER;
BEGIN
    v_dias :=
MI_EMPRESA.PKG_PROCESOS_BLOQUEOS.FN_CALCULAR_DIAS_MORA(:BLK_PV.ID_PUNTO_VENTA);
    :BLK_PV.IT_DIAS_MORA := v_dias;
END;
```

Resumen visual

text



Reglas y buenas prácticas

Regla	Por qué
No repitas el nombre del paquete en el procedimiento	PKG_BLOQUEOS.PR_BLOQUEAR_PUNTO_VENTA, no PKG_BLOQUEOS.PR_PKG_BLOQUEOS_BLOQUEAR
Usa prefijos PR_, FN_	Distingue visualmente procedimientos de funciones
Usa parámetros nombrados (=>)	Evita errores al cambiar el orden
Nombra los parámetros con p_	Sabes al instante que son parámetros y no variables locales
Nombra las variables locales con v_	Evita colisiones con nombres de columnas
Documenta cada procedimiento en la spec	Facilita el mantenimiento
Un paquete por área funcional	PKG_PROCESOS_BLOQUEOS, PKG_ARREGLOS_PAGO, PKG_REPORTES

Regla	Por qué
Cuerpo y spec separados	Puedes cambiar el body sin recompilar dependencias

¿Cuándo crear un paquete nuevo vs. agregar al existente?

Situación	Acción
Es una operación relacionada con bloqueos	Agrega al <code>PKG_PROCESOS_BLOQUEOS</code>
Es una operación de arreglos de pago	Crea <code>PKG_ARREGLOS_PAGO</code>
Es una utilidad general (fechas, strings)	Crea <code>PKG_UTILIDADES</code>
Son reportes	Crea <code>PKG_REPORTES</code>
Son reglas de negocio complejas	Crea un paquete por dominio (<code>PKG_CLIENTES</code> , <code>PKG_FACTURAS</code>)

Reglas para los GRANT (permisos)

Si Forms se conecta con un usuario distinto al dueño del paquete:

```
sql

-- Dar permiso al usuario de Forms
GRANT EXECUTE ON MI_EMPRESA.PKG_PROCESOS_BLOQUEOS TO USUARIO_FORMS;

-- Si es un paquete que se va a usar en vistas o triggers de BD
GRANT EXECUTE ON MI_EMPRESA.PKG_PROCESOS_BLOQUEOS TO PUBLIC;
```

Y para que Forms lo llame **sin el nombre del esquema**, crea sinónimos públicos:

```
sql

CREATE PUBLIC SYNONYM PKG_PROCESOS_BLOQUEOS FOR MI_EMPRESA.PKG_PROCESOS_BLOQUEOS;
```

Así desde Forms solo escribes:

sql

```
PKG_PROCESOS_BLOQUEOS.PR_BLOQUEAR_PUNTO_VENTA(...);
```

Resumen final

Escenario	Sintaxis
Sin parámetros	<code>ESQUEMA.PAQUETE.PROCEDIMIENTO;</code>
Con parámetros IN	<code>ESQUEMA.PAQUETE.PROCEDIMIENTO(p1, p2);</code>
Con parámetros nombrados	<code>ESQUEMA.PAQUETE.PROCEDIMIENTO(p1 => v1, p2 => v2);</code>
Con parámetros OUT	<code>ESQUEMA.PAQUETE.PROCEDIMIENTO(p1, p2, var_out);</code>
Función	<code>v := ESQUEMA.PAQUETE.FUNCION(param);</code>
Desde Forms	Igual, pero dentro del trigger del botón con <code>BEGIN ... END;</code>

Con esto ya tienes la estructura completa y las convenciones que usan las empresas para organizar su código PL/SQL en paquetes.

Para que otro usuario pueda acceder a los objetos que creas en Oracle, necesitas otorgar privilegios y, opcionalmente, crear sinónimos. A continuación, te explico cómo hacerlo para cada tipo de objeto.

🔑 Privilegios para Objetos de Oracle

Tablas, Vistas y Secuencias

- **Privilegios:** `SELECT`, `INSERT`, `UPDATE`, `DELETE` (para tablas/vistas) y `SELECT`, `ALTER` (para secuencias).
- **Ejemplo:**

sql

```
-- Otorgar permisos de solo lectura sobre una tabla
GRANT SELECT ON mi_esquema.mi_tabla TO otro_usuario;

-- Otorgar permisos de lectura y escritura
GRANT SELECT, INSERT, UPDATE, DELETE ON mi_esquema.mi_tabla TO otro_usuario;
```

Procedimientos, Funciones y Paquetes

- **Privilegio:** `EXECUTE`. Este permiso es único para todos ellos y permite ejecutarlos o usarlos en consultas.
- **Ejemplo:**

sql

```
-- Otorgar permiso para ejecutar un procedimiento
GRANT EXECUTE ON mi_esquema.mi_procedimiento TO otro_usuario;

-- Otorgar permiso sobre un paquete completo
GRANT EXECUTE ON mi_esquema.pkg_procesos_bloqueos TO otro_usuario;
```

Si necesitas ser más granular, puedes otorgar `EXECUTE` sobre un procedimiento específico dentro de un paquete.

Disparadores (Triggers)

- No se otorgan privilegios de objeto directamente sobre un trigger. El control se basa en **privilegios de sistema**:
- `CREATE TRIGGER`: Para crear un trigger en tu propio esquema sobre tus propias tablas.
- `CREATE ANY TRIGGER`: Para crear un trigger en cualquier esquema.

- **Importante:** El **dueño del trigger** debe tener privilegios directos (no a través de roles) sobre las tablas y procedimientos que el trigger utiliza en su lógica interna.

Jobs (Tareas Programadas con DBMS_SCHEDULER)

- **Privilegio de sistema:** `CREATE JOB` para crear tus propios jobs.
- **Privilegio de objeto:** Si usas un "Program" (objeto que define el ejecutable), necesitas el privilegio `EXECUTE` sobre él para que otros usuarios puedan asociarlo a sus jobs.
- **Ejemplo:**

sql

```
-- Dar permiso a un usuario para que use un "Program" existente en su Job
GRANT EXECUTE ON mi_esquema.mi_programa_scheduler TO otro_usuario;
```

Creación de Sinónimos

Un **sinónimo** es un alias para un objeto, lo que te permite referirte a él sin usar el prefijo del esquema (`mi_esquema.`).

- **Privilegio de sistema:** Necesitas `CREATE PUBLIC SYNONYM` para crear uno público, o `CREATE SYNONYM` para uno privado en tu esquema.
- **Ejemplo:**

sql

```
-- Crear un sinónimo público para un paquete
CREATE PUBLIC SYNONYM pkg_bloqueos FOR mi_esquema.pkg_procesos_bloqueos;

-- Crear un sinónimo privado para una tabla
CREATE SYNONYM empleados FOR mi_esquema.rh_empleados;
```

Un sinónimo público es visible para todos los usuarios. Un sinónimo privado solo es visible para el usuario que lo crea.

Buenas Prácticas

- **Principio de Mínimo Privilegio:** Otorga solo los permisos estrictamente necesarios para la tarea. Por ejemplo, si un formulario solo necesita leer datos, concédele únicamente `SELECT`.

- **Usa Roles:** Crea roles de base de datos (ej. `ROL_CONSULTAS`, `ROL_MANTENIMIENTO`) y agrupa los privilegios en ellos. Luego, asigna roles a los usuarios en lugar de permisos individuales. Esto simplifica enormemente la administración.
- **GRANT vs. GRANT ... WITH GRANT OPTION:** La opción `WITH GRANT OPTION` permite al usuario que recibe el privilegio otorgarlo a su vez a otros. Úsala con extrema precaución.

Si te surge alguna duda con un caso concreto de tu sistema, puedes consultarlo. ahora que te dije que usan un repositorio azure dev con git. una vez creado el formulario en oracle forms builder, lo compilo sin errores y para probarlo tendria que subirlo al repositorio git de azure. que archivos debo subir al servidor?. investiga algo de el software acselx de oracle , por lo menos necesito 3 paginas de informacion

El usuario necesita saber qué archivos subir al repositorio Git de Azure DevOps después de compilar un formulario en Oracle Forms Builder, y también requiere información sobre el software AcseIX. Para abordar esto, necesito buscar información sobre AcseIX y sobre las prácticas de despliegue en Oracle Forms con Azure DevOps. Voy a realizar varias búsquedas para cubrir ambos temas.

Found 58 web pages

Ahora que tengo los resultados de la búsqueda, necesito extraer la información relevante. Para AcseIX, los resultados 0, 1, 3 y 5 parecen prometedores. Para el despliegue de Oracle Forms, los resultados 0, 4, 5 y 6 son relevantes. Para obtener más detalles, abriré varias páginas.

Read 8 pages

[COLSECUROS Una compañía de Allianz](#)

[COTECSA](#)

[DESAROLLADOR ORACLE ACSEL/X](#)

[DESAROLLADOR ORACLE ACSEL/X | Eempleo.com - Postulación exitosa](#)

[Oracle Developer Cloud Service を使用した Oracle Forms での DevOps](#)

View All

La búsqueda ha proporcionado información sobre AcseIX y sobre el despliegue de Oracle Forms. Ahora necesito estructurar la respuesta. Primero, abordaré la pregunta sobre qué archivos subir al repositorio Git de Azure DevOps. Luego, proporcionaré la información sobre AcseIX que he encontrado. La respuesta debe ser detallada y práctica, con ejemplos de

comandos de Git y una explicación de los archivos que deben subirse. También debo asegurarme de incluir la información sobre AcselX de forma clara y organizada. Para tu primera pregunta, la respuesta es directa: **no debes subir el archivo compilado (.fmx) al repositorio de Git**. La práctica estándar es subir únicamente los archivos fuente (.fmb, .pll, .mmb) y dejar que un proceso automatizado (como Azure Pipelines) se encargue de compilar y desplegar. Esto evita conflictos y asegura que todos trabajen sobre la misma base de código.

¿Qué archivos subir a Azure DevOps?

Piensa en tu repositorio como la **bóveda de planos originales** de tu aplicación, no como el taller donde se construyen. Los archivos que debes versionar son los que tú editas:

Tipo de Archivo	Extensión	¿Qué es?
Formularios	.fmb	El código fuente de tu formulario.
Bibliotecas PL/SQL	.pll	El código fuente de tus bibliotecas de funciones.
Menús	.mmb	El código fuente de tus menús.
Reportes	.rdf	El código fuente de tus reportes.
Scripts SQL	.sql	Scripts para la base de datos.

Por el contrario, los archivos que **NUNCA** debes subir son los compilados (.fmx, .mmx, .plx, .rep). Para evitar subirlos por error, crea un archivo llamado .gitignore en la raíz de tu repositorio local y añade estas líneas:

text

```
# Ignorar archivos compilados de Oracle Forms
*.fmx
*.mmx
*.plx
*.rep
```

Flujo de trabajo correcto:

1. Creas o editas tu formulario `FOR_BLOQUEO_PV.fmb` en tu equipo local.
2. Lo compilas localmente para probarlo, generando `FOR_BLOQUEO_PV.fmx`. Este archivo **no lo subes**.
3. Haces commit y push **solo del archivo** `.fmb` y otros fuentes a Azure DevOps.

bash

```
git add forms/FOR_BLOQUEO_PV.fmb
git commit -m "feat: Agregar formulario de bloqueo de puntos de venta"
git push origin feature/bloqueo-pv
```

El despliegue real se hará luego copiando ese `.fmx` a la carpeta del servidor de Forms (`FORMS_PATH`) o usando un proceso de compilación automatizado.

■ AcseIX: El Core Asegurador sobre Oracle

AcseIX es un **sistema "core" para la industria de seguros**. En términos sencillos, es el software central que una aseguradora usa para gestionar todo su negocio, desde la creación de un producto hasta el pago de un siniestro.

- **¿Qué es?** Es una plataforma integral que soporta el ciclo de vida completo de una póliza de seguros, incluyendo emisión, siniestros, comisiones, cobranzas, reaseguros y coaseguros.
- **Tecnología:** Está construido sobre la pila tecnológica de Oracle. Específicamente, utiliza **Oracle Database** para los datos y **Oracle Forms/Reports** para la interfaz de usuario, en sus versiones **6i, 10g y 12c**.
- **¿Quién lo usa?** Es un sistema utilizado por aseguradoras en Latinoamérica, como Colseguros (Allianz) en Colombia, y es implementado y mantenido por consultoras como Inetum, COTECSA y VPD Proyectos y Sistemas.

Como desarrollador trabajando con AcseIX, tu día a día implicará:

- **Analizar y optimizar PL/SQL:** Crear y modificar procedimientos, funciones y paquetes para implementar o corregir lógica de negocio.

- **Mantener formularios y reportes:** Desarrollar y dar soporte a las pantallas y reportes en Oracle Forms y Reports.
- **Entender el modelo de datos:** Es crucial comprender las relaciones entre las tablas de AcselX para realizar cambios sin afectar otras áreas del sistema.

Si necesitas más detalles sobre el flujo de trabajo con Git o sobre algún aspecto específico de AcselX, puedes preguntar de nuevo.